Pico 2 Wireless Handheld Game Console

Project report

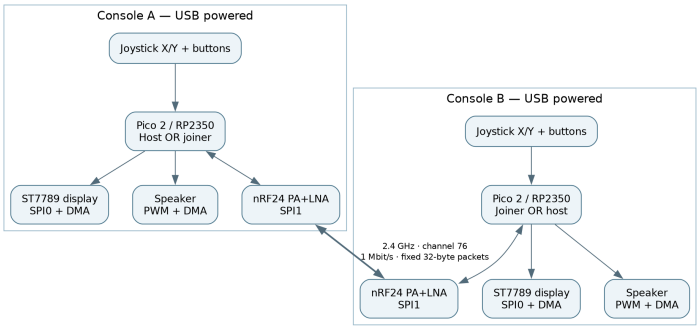
1. Overview

We built two standalone handheld consoles around the Raspberry Pi Pico 2. Games run locally. The wireless link carries inputs and compact game state, not video.

Either console can host a match. The host runs the authoritative game logic. The joiner sends controls and renders the state received from the host.

At a glance	Configuration
Processing and display	Non-wireless Pico 2; 2.8-inch ST7789 display; 320×240 landscape; 30 FPS target
Wireless link	nRF24L01+ PA+LNA transceiver; channel 76; 1 Mbit/s
Multiplayer updates	30 Hz host state publication; local rendering and sound on both consoles

Each console has a joystick, seven buttons and a SmartElex Digital Speaker Module. Both units are USB-powered.



Terms used in this report

Term	Meaning
DMA	Direct Memory Access: hardware transfers data without a CPU loop for each item.
PWM	Pulse-Width Modulation: varying a digital signal's duty cycle to produce the audio waveform.
ACK / CRC	Acknowledgement / Cyclic Redundancy Check: reception feedback and packet-corruption detection.
Snapshot / tick	A description of game state / the host's game-update counter.
FNV-1a	The hash function used to derive our compact session ID.

> **Timing terms:** RF airtime is the time a packet occupies the radio channel. Round-trip time covers a message and its reply. Button-to-screen latency covers an input through to its visible result.

2. Games

Game	Basic idea
Flappy Bird	Fly through pipe gaps; solo and two-player matches.
Pong	Return balls with a paddle; four levels add speed changes and multiball. Solo uses a full-height opposing wall.
Grid Blasters	Move, shoot and dash in an arena; five randomly selected maps.
Race Duel	Avoid traffic, collect boosts and outlast the opponent on a shared scrolling road.

3. Why nRF for this project?

We use the nRF24L01+ PA+LNA module: a packet transceiver with an external Power Amplifier and Low-Noise Amplifier.

Our requirement is a direct link between two dedicated consoles. Messages are small and frequent. We do not need phone connectivity, IP networking or large file transfers.

Consideration	nRF24L01+	Wi-Fi	Bluetooth Low Energy (BLE)
Integration	Small SPI driver and our connection manager	Supported wireless hardware and networking stack	Supported wireless hardware and BLE stack
Small game messages	Compact 32-byte payloads	Much greater capacity than these games require	Also suitable for small messages
Timing	Firmware initiates packets when the radio is available	Medium access, stack queues and power saving affect timing	Connection-event scheduling and parameters affect timing
Setup	Our direct Host/Join exchange	Requires a supported network or direct-link arrangement	Advertising, connection roles and parameters
External compatibility	Dedicated matched consoles	Standard computer/network connectivity	Standard phone/peripheral connectivity

The installed boards are **Pico 2, not Pico 2 W**. Pico 2 W has built-in Wi-Fi/Bluetooth; the non-W boards do not. Using nRF keeps the existing boards and gives our firmware direct control over packet scheduling and reply timing, without a Wi-Fi or BLE connection stack.

At 1 Mbit/s, a 32-byte payload occupies about **0.33 ms of RF airtime**, including our radio framing. Our firmware controls when the exchange starts. This suits frequent control and state messages.

nRF is therefore a good fit for this specific workload. It is not inherently faster or more power-efficient than every Wi-Fi or BLE configuration. Both alternatives can work without internet access, and BLE needs no router.

4. Hardware and connections

The ST7789 display uses SPI0; the radio uses SPI1. Separate buses keep long display transfers off the radio's SPI interface. The SmartElex module amplifies the PWM audio signal from the Pico.

PA+LNA and supply decoupling

The PA increases transmit signal power; the LNA amplifies the received signal. They can improve signal margin and range, but add supply load. Payload size and packet speed remain unchanged.

Transmit bursts create rapid load changes. Supply wires have resistance and inductance, so the module voltage can briefly dip. A local capacitor across VCC and GND supplies short current pulses.

A small ceramic capacitor handles fast disturbances; a bulk capacitor supports slower changes. Place them close to the module with short connections. Decoupling complements a suitable supply and grounding; it does not replace them.

Pin allocation

Both consoles use this allocation. GP numbers are GPIO numbers, not header positions.

Connection	Pico GPIO	Header pin
Display CS — SPI0	GP17	22
Display SCK — SPI0	GP18	24
Display SDI / MOSI — SPI0	GP19	25
Display DC	GP20	26
Display RESET	GP21	27
Display SDO / MISO — connected, unused by normal transfers	GP16	21
Radio CE	GP11	15
Radio MISO — SPI1	GP12	16
Radio CSN — SPI1	GP13	17
Radio SCK — SPI1	GP14	19
Radio MOSI — SPI1	GP15	20
A / B / C / D — Up / Right / Down / Left	GP5 / GP8 / GP4 / GP7	7 / 11 / 6 / 10
E / F / K — Pause / Menu / Joystick click	GP3 / GP6 / GP2	5 / 9 / 4
Joystick X / Y	GP27 ADC1 / GP26 ADC0	32 / 31
Speaker VIN	GP22	29
Radio and speaker supply	3V3 OUT	36
Display VCC and LED supply	VBUS	40
Common ground	GND	3, 8, 13, 18, 23, 28 or 38

Radio IRQ is disconnected; firmware polls status through SPI. Buttons use internal pull-ups and 20 ms debounce. Joystick X is uninverted; Y is inverted in `controls.cpp`.

5. Code and development

The console framework and drivers are shared. Each game owns its rules, rendering, audio triggers and game-body encoding.

File or group	Responsibility
`src/main_pico.cpp`	Hardware startup and the main input/game/display/radio/audio loop.
`src/console.cpp`	Game selection, menus, pause and the shared sound setting.
`include/game.h`	Common game callbacks and input contract.
`src/controls.cpp`	Buttons, debounce, ADC axes and inversion.
`src/graphics.cpp`	Framebuffers, text, shapes and sprites.
`src/display.cpp`	ST7789 setup and SPI0/DMA transfers.
`src/radio.cpp`	nRF register setup, payload transmission and reception.
`src/multiplayer.cpp`	Connection screen, pairing, common packet handling and fragment assembly.
`src/speaker.cpp`	PWM/DMA playback and sound priorities.
`src/game_*.cpp`, `include/*_assets.h`	Individual games and compiled artwork/sound data.
`src/main_sim.cpp`	Desktop simulator using shared console and game code.
`CMakeLists.txt`, `tests/`	Build/game discovery and separate hardware diagnostics.

Build and flash

Firmware prerequisites are CMake, the ARM GNU toolchain (`arm-none-eabi-gcc`) and the Pico SDK. The simulator needs a native C++ compiler and SDL2 development libraries.

Set `PICO_SDK_PATH` to the installed Pico SDK, then build the firmware:

```
export PICO_SDK_PATH=/path/to/pico-sdk
cmake -S . -B build_pico -DPICO_BUILD=ON -DCMAKE_BUILD_TYPE=Release
cmake --build build_pico --target pico_game -j4
```

Flash `build_pico/pico_game.uf2` through BOOTSEL, or use picotool. Its `--ser` option selects a board by its USB serial number. `-v` verifies the write and `-x` starts the firmware.

The SDL2 simulator supports local game development without flashing. Build it with `cmake -S . -B build_sim`, then `cmake --build build_sim -j4`. Run `./build_sim/sim`.

It shares gameplay code, but does not reproduce physical radio delivery or panel scanning.

6. Display and audio

Display

The 2.8-inch ST7789 module is used in 320×240 landscape mode. RGB565 stores each pixel in two bytes. The application targets 30 FPS; the panel is configured for nominal 60 Hz refresh.

Panel scanning is independent of application rendering. A frame may remain visible for more than one scan.

We use two complete framebuffers:

1. The CPU draws into one buffer.
2. DMA sends the other through SPI0.
3. Buffer roles swap only after a new transfer is accepted.

All four games transfer full frames at a requested 75 MHz SPI clock. With the 150 MHz peripheral clock, this is the SPI hardware's maximum output: its minimum clock divider is two. This exceeds the published ST7789V serial-write limit: a 66 ns minimum cycle, or about 15.15 MHz. It works on our units; reduce the clock if artifacts appear.

A full RGB565 frame is 153,600 bytes. At exactly 75 MHz, its pixel transfer takes a calculated 16.38 ms, before command overhead. A nominal 60 Hz panel scan takes 16.67 ms. These independent operations have nearly equal periods, so their relative timing drifts.

Double buffering protects DMA source memory, but does not swap the panel's image atomically. A scan can cross from old to newly written pixels, creating a tear boundary that moves as timing drifts. Its diagonal appearance also depends on the landscape scan and write directions.

Audio

The SmartElex Digital Speaker Module plays local effects for flaps, shots, points, crashes and results. Samples are compiled into game assets and remain in flash.

The driver uses PWM output and DMA playback at 24 kHz. Playback is single-voice: an equal- or higher-priority effect can replace the active sound.

The shared Sound setting applies across games during the current boot. Each console keeps its own setting. Normal gameplay carries no audio stream over the radio.

7. Wireless protocol

Radio link and addressing

All games share channel 76, 1 Mbit/s, five-byte addresses and fixed 32-byte application payloads. The nRF adds its own preamble, destination, control field and two-byte CRC.

Hardware auto-ACK is enabled, with up to three retransmissions and a configured 500 µs retry delay.

Radio framing and application packet layouts

Over the air — radio-generated fields surround the 32-byte application payload

Preamble	Destination	Control	Application payload	CRC
1 byte	5 bytes	9 bits	32 bytes	2 bytes

Pairing payload — address is NOT repeated inside this payload

P2MP	Ver.	Type	Game	Host token	Joiner token	Sequence	Zero
0–3	4	5	6–7	8–15	16–23	24–27	28–31

Gameplay payload — 8-byte shared header + 24-byte game body

D7	Ver./type	Session ID	Host tick	Game body
0	1	2–5	6–7	8–31

Role	Receive address	Transmit destination
Host	`A7 D3 C1 5A 02`	`B6 D3 C1 5A 02`
Joiner	`B6 D3 C1 5A 02`	`A7 D3 C1 5A 02`

Addresses are fixed application constants, shown in driver byte-array order. They are not serial numbers or generated session IDs. Their exact values are arbitrary; distinct roles and matching transmitter/receiver settings matter. Our driver requires different first bytes.

The RF signal propagates through the air. Address filtering determines which frames a radio accepts. The address belongs to the radio frame, outside our 32-byte payload.

Pairing

Both consoles select the same game. Host starts announcements; Join listens for the first compatible Beacon. There is no host-selection list.

Every pairing message uses the layout below. Offsets start at zero. Multi-byte numbers use little-endian order: the lowest byte is stored first.

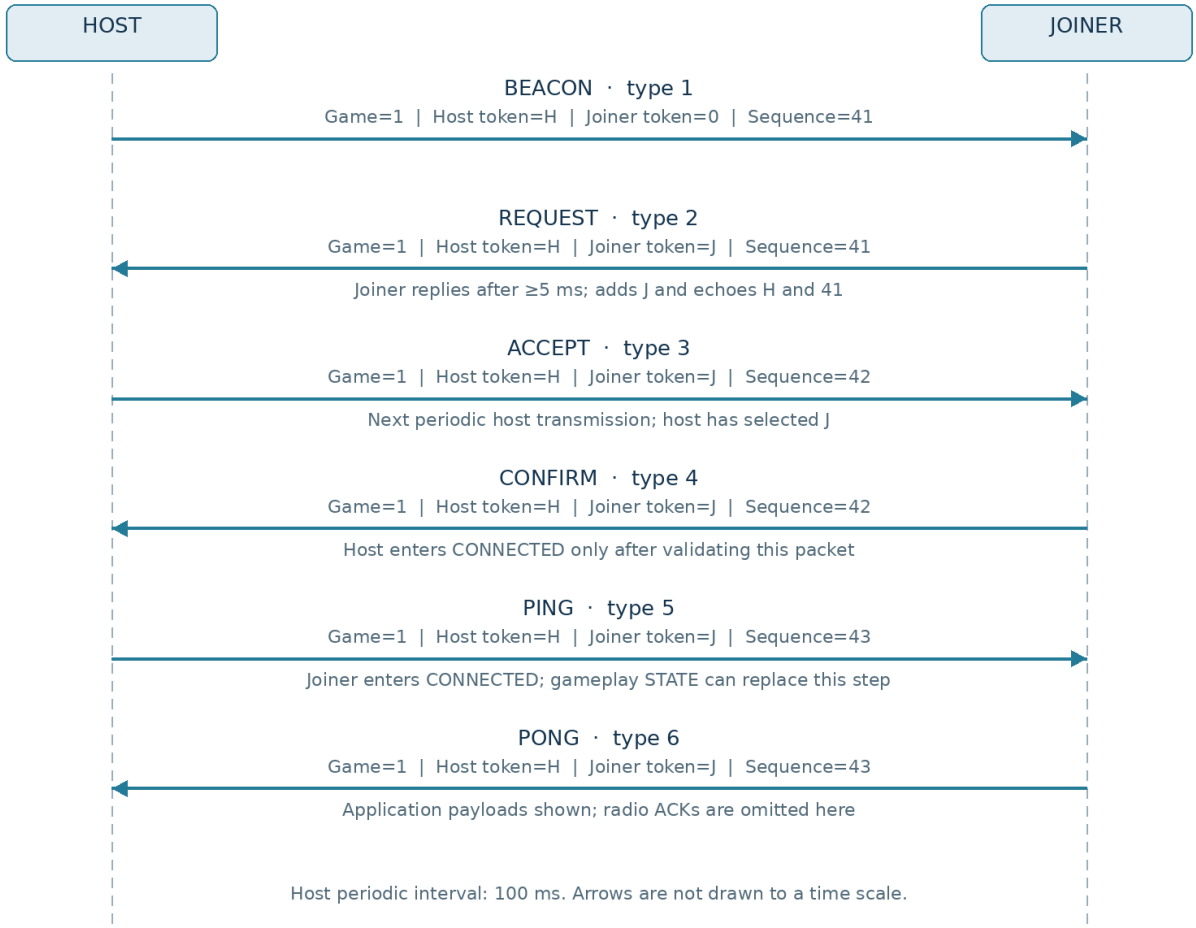
Bytes	Field	Meaning and value source
0–3	`P2MP` marker	Fixed project-specific pairing marker, not a standard protocol.

Bytes	Field	Meaning and value source
4	Version	Fixed value 1.
5	Message type	Fixed codes: Beacon=1, Request=2, Accept=3, Confirm=4, Ping=5, Pong=6.
6–7	Game ID	Fixed per game; both consoles must match.
8–15	Host token	Nonzero 64-bit value generated at role start from local microsecond uptime and a counter.
16–23	Joiner token	Generated in the same way by the joiner; zero in a Beacon.
24–27	Exchange sequence	Host counter starting at 1. Increments for each periodic message; wraps and skips zero. Replies echo it.
28–31	Padding	Fixed zeros.

Game IDs are independent of menu order. Flappy and Race Duel use numeric IDs `0x0001` and `0x0004`. Pong and Grid Blasters use mnemonic values `0x5047` and `0x4742`—`PG` and `GB` when read as hexadecimal character codes.

The diagram uses token placeholders H/J and illustrative sequences 41–43. Radio ACKs are omitted to show the application handshake clearly.

Pairing sequence — illustrative exchange numbers



1. **Beacon:** host advertises the game, token H and sequence 41. Joiner token is zero.
2. **Request:** joiner echoes H and 41, adding token J. The host validates the reply and selects J.
3. **Accept:** the next periodic host message carries H, J and a new sequence, illustrated as 42.
4. **Confirm:** joiner echoes H, J and 42. Valid reception makes the host connected.
5. **Ping/Pong:** a newer Ping completes the joiner's connected state. It replies with Pong. A valid gameplay STATE can complete this transition instead.

The host's periodic interval is 100 ms. Joiner replies follow the common 5 ms response policy explained with its measured results in Section 8.

Gameplay protocol

The host owns physics, collisions, randomness and scoring. It publishes STATE snapshots at 30 Hz. The joiner renders them and returns INPUT at least 5 ms after accepting a complete STATE, following the reply-delay policy explained in Section 8.

Both players mark Ready before a three-second countdown. Host timeline ticks continue through Ready, pause and results, even when physics is frozen.

Bytes	Field	Meaning and value source
0	`D7` marker	Fixed gameplay marker.
1	Version/ type	Fixed codes: `0x11` STATE, `0x12` INPUT, `0x13` first STATE half, `0x14` second half.
2–5	Session ID	32-bit value derived using FNV-1a from both tokens and the game ID. Both consoles calculate it.
6–7	Host tick	Counted 16-bit game-update number, wrapping after 65,535. INPUT echoes the latest accepted STATE tick.
8–31	Game body	24 bytes allocated by the game for its state, inputs and command/event acknowledgements.

The type byte stores version 1 in its upper four bits. The session ID identifies the pairing attempt without repeating both full tokens. Tokens are deterministic generated values, not hardware-random credentials.

Games define their own 24-byte bodies. The connection manager transports them without interpreting game physics. Command/event numbers are assigned when actions are created and retained when repeated.

Gameplay exchange — STATE and INPUT



Two acknowledgement layers serve different purposes:

- **Hardware ACK:** automatic feedback that a radio received the frame. It does not confirm game-command execution. An ACK failure does not prove the original payload was lost.

- **Application ACK:** a field in a later game packet confirms a command or event was handled. Its sender can then stop repeating that action.

For example, Ready command 7 repeats in later INPUT packets until STATE reports it handled. The number prevents repetition from creating a second action.

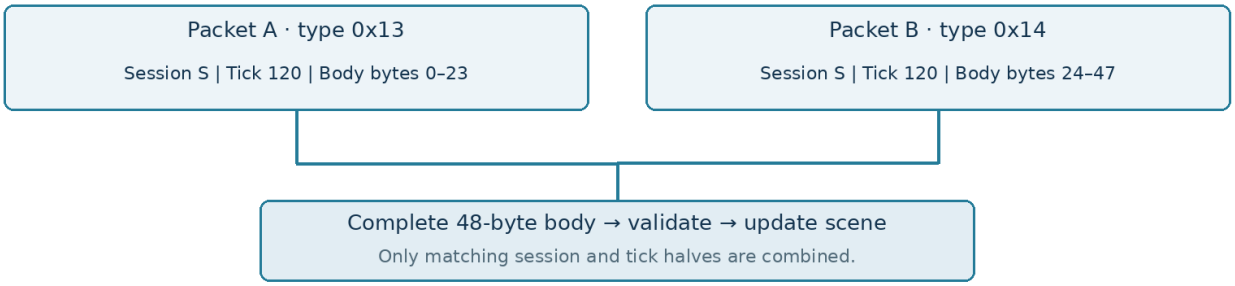
The host does not wait for every INPUT. It continues with usable controls under each game's expiry policy. New snapshots replace older unsent snapshots; they do not form a growing queue. Duplicate and older accepted ticks are rejected.

Two-packet snapshots

Race Duel uses the supported 48-byte STATE format. Two normal radio packets carry its body:

- `0x13`: header plus body bytes 0–23.
- `0x14`: header plus body bytes 24–47.

Race Duel — assembling a two-packet snapshot



One half alone: keep the previous complete scene; no partial update or INPUT reply.

Both halves must match in session and tick. Only a complete validated pair updates the scene and schedules INPUT. A newer pair replaces an incomplete older assembly.

After the second fragment's observed TX completion, the host reserves 10 ms in receive mode. This covers the common reply delay and a transmission/retry allowance. INPUT still uses an ordinary 24-byte body.

This supports a two-fragment STATE, not an arbitrary-size stream.

Link loss

Only valid accepted peer traffic refreshes connection activity. Duplicate, rejected or incomplete snapshots do not extend it.

After five seconds of silence, the game aborts and the connection screen returns. Rejoining is manual. Initial hosting/searching has no timeout.

Short losses use different policies: new snapshots replace old scenes, held controls expire, and numbered commands/events repeat until acknowledged.

8. Resources and measured results

Memory

Four-game Release allocations, on Pico 2's 520 KiB SRAM and 4 MiB flash:

Item	Bytes
Flash image, including code and assets	250,328
Static main-SRAM extent	319,920
Two framebuffers, included above	307,200
Compiled sound samples, included in flash	144,422

Processing-time results

Full-frame, sound-enabled wireless runs used normal radio settings. Two 50-second scripted runs per game swapped host roles. Rounds include countdowns/rematches; Grid was low-action.

Values are **host / joiner elapsed-time percentages**, not CPU-cycle utilization. Audio service covers main-loop maintenance; sound triggers remain in game updates.

Game	Drawing	Radio	Game updates	Audio service	Sleep
Flappy	11.49 / 11.47	2.95 / 3.08	0.22 / 0.13	0.06 / 0.04	83.62 / 83.60
Pong, level 4	4.91 / 4.90	2.90 / 2.93	0.29 / 0.11	0.06 / 0.05	90.29 / 90.48
Grid Blasters	6.87 / 6.92	2.99 / 3.10	0.66 / 0.15	0.06 / 0.04	87.75 / 88.21
Race Duel	18.30 / 18.30	4.49 / 3.27	0.35 / 0.08	0.06 / 0.05	75.13 / 76.69

Other work: controls 0.83–0.89%; display/logging/bookkeeping 0.68–0.83%. Games averaged 30 updates/s and 29.3–29.7 completed frames/s.

Mean draw: Flappy 3.87 ms; Pong 1.67 ms; Grid 2.35 ms; Race 6.20 ms. Mean polling was 759–903 Hz; the largest gap was 10.28 ms in Race. Observed transfers averaged 18.3–18.6 ms, including completion-poll delay. DMA overlaps CPU work.

Reply-delay comparison

The echo comparison used 1 Mbit/s, 32-byte packets and ACK/retries. We tested 0.5 ms and 5 ms reply gaps in both orders.

Reply gap	Missed / attempted exchanges	Missing
0.5 ms	152 / 4,000	3.8%
5 ms	1 / 4,000	0.025%

With the 5 ms wait, successful exchanges averaged 7.02 ms RTT. The remaining 2.02 ms covers both transmissions, turnaround, ACK/retries and processing.

Early replies can overlap ACK/retries. Waiting reduced missed replies, so we use a minimum 5 ms application reply delay.

Radio delivery and payload demand

The standalone delivery test used channel 76, 1 Mbit/s, 32-byte packets, 30 Hz and ACK + three retries, with no display or game running. Delivery was 998/1,000 packets A→B and 1,000/1,000 B→A.

Calculated at 30 exchanges/s: one-packet games carry 1,920 payload bytes/s; Race Duel 2,880 bytes/s, excluding radio framing, ACKs and retries.